دليل مناقشة مشروع التحليل العددي

مرجع شامل للاستعداد للمناقشة الشفهية والبرمجية مع الدكتور (شرح الكود، الماتلاب، والواجهات)

اللغة: Python 3 + Tkinter | المرجع الحسابي: MATLAB Codes | التصميم: Google Material Design 3

المقرر: التحليل العددي للأنظمة غير الخطية

1. هيكل المشروع وهندسة تدفق البيانات بين الواجهات الثلاث

تم تصميم المشروع وفق مبدأ **فصل الاهتمامات (Separation of Concerns)**؛ حيث تم عزل المنطق الحسابي والرياضي في مجلد `/methods` تماماً عن عناصر واجهات المستخدم الرسومية في مجلد `/gui` ومساعدات التحليل في `/utils`.

أ. الهيكل التنظيمي للملفات في المجلد `/project`

الملف / المجلد	المسؤولية البرمجية	تفاصيل المحتوى والدور في المشروع
main.py	نقطة الدخول (Entry Point)	تهيئة مسارات النظام وتشغيل النافذة الرئيسية الأولى `MainWindow`.
gui/main_window.py	الواجهة الأولى (قائمة الطرق)	عرض قائمة الطرق الخمس مرقمة ببطاقات M3، وإدارة قاموس `METHODS` المركزي.
gui/input_window.py	الواجهة الثانية (إدخال البيانات)	بناء حقول المدخلات ديناميكياً بحسب الطريقة، فحص المدخلات، واستدعاء الخوارزميات.
gui/result_window.py	الواجهة الثالثة (عرض النتائج)	عرض جدول التكرارات كجدول السبورة بدقة 6 خانات، وإبراز قيمة الجذر وعدد التكرارات.
gui/theme.py	نظام المظهر (M3 Design)	لوحة الألوان الدلالية، أوزان الخطوط، وأنماط الأزرار والجداول في `ttk.Style`.
/methods	الخوارزميات الحسابية (5 طرق)	ملفات بايثون نقية منفصلة عن الواجهة تحاكي دوال الماتلاب المعتمدة لدى الدكتور.
utils/parser.py	المحلل الرياضي الرمزي	تحويل نص الدالة بالضرب الضمني والأسس إلى دوال عددية واشتقاق $f'(x)$ عبر `sympy`.
/matlab_reference	التوثيق المرجعي للماتلاب	ملف `README.md` يضم الأكواد الأصلية في ماتلاب للمطابقة أثناء المناقشة.

ب. مخطط تدفق البيانات والتنقل بين الواجهات (Data Flow & Window Lifecycle)



دورة حياة النوافذ: عند الانتقال للأمام، تُخفى النافذة السابقة بـ `withdraw()` للتركيز. عند النقر على "رجوع للمدخلات" في الواجهة الثالثة، تُغلق نافذة النتائج وتُعاد نافذة الإدخال بـ `deiconify()` محتفظة بالبيانات. والضغط على "القائمة الرئيسية" يُغلق النوافذ الفرعية ويُعيدك نظيفاً للنافذة الأولى دون ترك أي نوافذ معلقة في الذاكرة.

2. الخوارزميات الخمس ومقارنة أسطر كود الماتلاب بالبايثون

1. طريقة التنصيف (Bisection Method) - الملف methods/bisection.py

فكرة الطريقة: حصر الجذر داخل فترة مغلقة $[a, b]$ يتغير فيها إشارة الدالة ($f(a) \cdot f(b) < 0$)، ثم تنصيف الفترة في كل تكرار ($p = \frac{a+b}{2}$) واستبدال أحد الطرفين للحفاظ على إحاطة الجذر وتقليص طول الفترة إلى النصف في كل خطوة.

$$p = \frac{a + b}{2}, \quad \text{Condition: } f(a) \cdot f(b) < 0, \quad \text{Error: } \frac{|b - a|}{2} \le \varepsilon$$

#	كود الماتلاب الأصلي للدكتور: bisc(f, a, b, n, e)	كود البايثون المقابل في methods/bisection.py

1	`function bisc(f, a, b, n, e)`
	`i = 1;`
		`def bisection_method(f, a, b, tol=1e-4,`
		`max_iter=50):`
		`    table, i, p_prev = [], 1, None`

2	`while (abs(a - b) > e && i <= n)`	`while i <= max_iter:`
3	`    p = (a + b) / 2;` `    if f(p) == 0, break; end`	`    p = (a + b) / 2.0; fp = f(p)` `    if abs(fp) < 1e-14 or error <= tol: break`

4	`    if f(a) * f(p) < 0`
	`        b = p;`
	`    else`
	`        a = p;`
	`    end`
	`    i = i + 1;`
		`    if fa * fp < 0:`
		`        b = p`
		`    else:`
		`        a = p`
		`    p_prev, i = p, i + 1`

5	`disp(['الجذر التقريبي: ', num2str(p)]);`	`return table, p`

2. طريقة الموضع الخاطئ (False Position / Regula Falsi) - الملف methods/false_position.py

فكرة الطريقة: طريقة حصر تشبه التنصيف، لكن بدلاً من تنصيف الفترة حسابياً بالمنتصف، نصل بين النقطتين $(a, f(a))$ و $(b, f(b))$ بوتر مستقيم، ونأخذ نقطة تقاطعه مع محور السينات كتقدير جديد للجذر، مما يعطي تقارباً أسرع نحو الجذر من التنصيف العادي.

$$p = b - \frac{f(b)(b - a)}{f(b) - f(a)} = \frac{a\,f(b) - b\,f(a)}{f(b) - f(a)}, \quad \text{Error: } |p_k - p_{k-1}| \le \varepsilon$$

#	كود الماتلاب الأصلي للدكتور: false_position	كود البايثون المقابل في methods/false_position.py
1	`p = b - (f(b)*(b - a)) / (f(b) - f(a));` `while (abs(f(p)) > e && i <= n)`	`denom = fb - fa` `p = b - (fb * (b - a)) / denom`

2	`    if f(a) * f(p) < 0`
	`        b = p;`
	`    else`
	`        a = p;`
	`    end`
		`    if fa * fp < 0:`
		`        b = p`
		`    else:`
		`        a = p`

3	`    p = b - (f(b)*(b - a)) / (f(b) - f(a));` `    i = i + 1;`	`    error = abs(p - p_prev)` `    p_prev, i = p, i + 1`

3. طريقة القاطع (Secant Method) - الملف `methods/secant.py`

فكرة الطريقة: طريقة مفتوحة تشبه نيوتن رافسون، لكنها لا تتطلب حساب المشتقة تحليلياً؛ حيث تستبدل مشتقة المنحنى بميل القاطع المار بآخر نقطتين تقريبيتين x_{k-1} و x_k، ولا تشترط حصر الجذر بين إشارتين مختلفتين.

$$x_{k+1} = x_k - \frac{f(x_k)(x_k - x_{k-1})}{f(x_k) - f(x_{k-1})}, \quad \text{Error: } |x_{k+1} - x_k| \le \varepsilon$$

#	كود الماتلاب الأصلي: secant(f, x0, x1, n, e)	كود البايثون المقابل في methods/secant.py
1	`p = x1 - (f(x1)*(x1 - x0)) / (f(x1) - f(x0));` `while (abs(p - x1) > e && i < n)`	`denom = f1 - f0` `p = curr_x1 - (f1 * (curr_x1 - curr_x0)) / denom`
2	`x0 = x1;` `x1 = p;`	`curr_x0 = curr_x1` `curr_x1 = p`
3	`p = x1 - (f(x1)*(x1 - x0)) / (f(x1) - f(x0));` `i = i + 1;`	`error = abs(p - curr_x1)` `i += 1`

4. طريقة نيوتن - رافسون (Newton-Raphson Method) - الملف `methods/newton.py`

فكرة الطريقة: أسرع الطرق العددية وأعلاها رتبة تقارب (تقارب تربيعي $O(h^2)$)؛ تعتمد على رسم مماس المنحنى عند النقطة الابتدائية x_k واستخدام نقطة تقاطع المماس مع محور السينات كنقطة تالية، بالاعتماد على المشتقة الأولى $f'(x)$.

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \quad \text{Condition: } f'(x_k) \ne 0, \quad \text{Error: } |x_{k+1} - x_k| \le \varepsilon$$

#	كود الماتلاب: newton(f, df, x0, n, e)	كود البايثون المقابل في methods/newton.py
1	`while (i <= n)` `  if df(x0) == 0, error('Derivative is 0'); end`	`while i <= max_iter:` `  if abs(dfx) < 1e-15: raise ValueError("المشتقة تساوي صفراً")`
2	`  x1 = x0 - f(x0) / df(x0);` `  if abs(x1 - x0) <= e, break; end`	`  next_x = curr_x - (fx / dfx)` `  if abs(fx) < 1e-14 or error <= tol: return table, next_x`
3	`  x0 = x1;` `  i = i + 1;`	`  curr_x = next_x` `  i += 1`

5. طريقة النقطة الثابتة (Fixed Point Iteration) - الملف `methods/fixed_point.py`

فكرة الطريقة: إعادة صياغة المعادلة $f(x) = 0$ جبرياً بالشكل $x = g(x)$، ثم البدء بنقطة ابتدائية x_0 والتعويض المتسلسل $x_{k+1} = g(x_k)$ حتى تثبت القيمة، ويشترط لتقاربها أن تكون القيمة المطلقة للمشتقة أقل من الواحد $|g'(x)| < 1$.

$$x_{k+1} = g(x_k), \quad \text{Condition for Convergence: } |g'(x)| < 1, \quad \text{Error: } |x_{k+1} - x_k| \le \varepsilon$$

#	كود الماتلاب: fixed_point(g, x0, n, e)	كود البايثون المقابل في methods/fixed_point.py

1	`while (i <= n)`
	`  x1 = g(x0);`
		`while i <= max_iter:`
		`  next_x = g(curr_x)`
		`  if abs(next_x) > 1e12: raise ValueError("الدالة تتباعد")`

2	`  if abs(x1 - x0) <= e, break; end`	`  error = abs(next_x - curr_x)` `  if error <= tol: return table, next_x`
3	`  x0 = x1;` `  i = i + 1;`	`  curr_x = next_x` `  i += 1`

3. شرح ملفات واجهات المستخدم في مجلد `gui/` ووظائفها الأساسية

الملف	الدوال والفئات الرئيسية	الوظيفة والمهمة التقنية
gui/main_window.py (الواجهة 1)	▪ `class MainWindow(tk.Tk)` ▪ `METHODS: Dict` ▪ `_create_method_card()` ▪ `open_input_window()`	- تمثل النافذة الأولى للتطبيق وتعرّف قاموس `METHODS` الموحد. - توليد بطاقات الطرق الخمس التفاعلية مرقمة بأسلوب M3 مع تأثيرات المرور. - فتح شاشة الإدخال وإخفاء النافذة الرئيسية لتنظيم التركيز.
gui/input_window.py (الواجهة 2)	▪ `class InputWindow(tk.Toplevel)` ▪ `_build_dynamic_inputs()` ▪ `on_calculate()` ▪ `open_results_window()` ▪ `on_back_to_main()`	- بناء حقول الإدخال تلقائياً (a, b أو x0, x1 أو x0 مع المشتقة التلقائية). - فحص صحة المدخلات ومنع القيم الفارغة أو غير العددية. - تحويل نص الدالة بالـ Parser واستدعاء الخوارزمية وعرض الأخطاء بنافذة تنبيه دون انهيار. - إخفاء نافذة الإدخال وفتح نافذة النتائج.
gui/result_window.py (الواجهة 3)	▪ `class ResultWindow(tk.Toplevel)` ▪ `_build_treeview()` ▪ `on_back_to_inputs()` ▪ `on_back_to_main()`	- عرض جدول التكرارات في `ttk.Treeview` بأعمدة السبورة وأشرطة تمرير. - تنسيق الأرقام بدقة 6 خانات عشرية وتلوين متبادل للصفوف. - إبراز السطر المطلوب: **"الجذر هو x = ..."** مع عدد التكرارات. - أزرار مرنة للعودة للمدخلات أو للرئيسية مع دعم تمدد الجدول مع حجم النافذة.
gui/theme.py (نظام التصميم)	▪ `setup_theme(root)` ▪ `COLOR_PRIMARY, COLOR_BG ...` ▪ `FONT_HEADER, FONT_BUTTON ...`	- تطبيق معايير Material Design 3 الرسمية للألوان والأبعاد. - ضبط أنماط ttk (Clam theme) للأزرار والحقول والجداول. - توحيد الخط العربي `Segoe UI` لضمان مظهر احترافي عالي الجودة.

4. الأسئلة المحتملة من الدكتور في المناقشة الشفهية وإجاباتها النموذجية

هندسة البرمجيات **س1: لماذا قمت بفصل الخوارزميات في مجلد methods/ عن واجهات gui/؟**

الإجابة: تطبيقاً لمبدأ *فصل الاهتمامات (Separation of Concerns)* وبنية *MVC (Model-View-Controller)*؛ الخوارزميات في `methods/` تمثل المنطق الرياضي الصرف (Pure Math Logic) دون أي اعتماد على الواجهة، مما يجعلها مطابقة لكود الماتلاب، قابلة للاختبار الآلي، وقابلة لإعادة الاستخدام في أي بيئة أخرى دون تعديل، بينما يقتصر دور `gui/` على العرض وتلقي المدخلات.

التحليل العددي **س2: كيف يُحسب مقدار الخطأ (Error) في كل طريقة وما معناه؟**

الإجابة:

- **التنصيف:** يُحسب الخطأ بنصف طول الفترة $\frac{|b - a|}{2}$ أو $|p_k - p_{k-1}|$؛ لأن الجذر الفعلي محصور حتماً داخل الفترة فيكون أقصى بعد بين التقدير والجذر هو نصف طول الفترة.
- **الموضع الخاطئ والقاطع ونيوتن والنقطة الثابتة:** يُحسب بالفرق المطلق بين تقريبين متتاليين $|x_{k+1} - x_k|$ (الخطأ المطلق للتكرار)، أو بقيمة الدالة عند النقطة $|f(x)| \approx 0$.

شروط الخوارزمية **س3: متى تتوقف الخوارزمية عن التكرار (Stopping Criteria)؟**

الإجابة: تتوقف الخوارزمية عند تحقق أحد ثلاثة شروط:

1. **الوصول للجذر التام:** عندما تكون قيمة الدالة مساوية تماماً للصفر $$f(p) = 0$$ (أو أقل من 10^{-14}).
2. **تحقق الدقة المطلوبة:** عندما يصبح الخطأ المحسوب أقل من أو يساوي الدقة المدخلة ($Error \le \varepsilon$).
3. **بلوغ الحد الأقصى للتكرارات:** عندما يصل العداد $i > N$ لحماية البرنامج من الدخول في حلقة لانهائية (Infinite Loop) في حال تباعدت الطريقة.

مقارنة **س4: ما هو الفرق الجوهري بين طريقة نيوتن رافسون وطريقة القاطع؟**

الإجابة: طريقة **نيوتن رافسون** تحتاج نقطة بداية واحدة x_0 لكنها *تشترط وجود المشتقة التحليلية $f'(x)$* وتقاربها تربيعي وسريع جداً ($O(h^2)$)، بينما طريقة **القاطع** تحتاج نقطتين ابتدائيتين x_0, x_1 وتستبدل المشتقة بميل القاطع المار بالنقطتين $\frac{f(x_1)-f(x_0)}{x_1-x_0}$ للتغلب على صعوبة اشتقاق بعض الدوال المعقدة، ورتبة تقاربها ≈ 1.618 (أبطأ قليلاً من نيوتن لكن أسرع من التنصيف).

شروط الفشل **س5: متى تفشل طريقة نيوتن - رافسون؟ وكيف عالجنا ذلك؟**

الإجابة: تفشل إذا كانت المشتقة عند أي تكرار مساوية للصفر ($f'(x_k) = 0$)؛ لأن القانون يقسم على المشتقة، وهندسياً يعني ذلك أن مماس المنحنى أفقي موازٍ لمحور السينات فلا يتقاطع معه. عالجنا ذلك في `methods/newton.py` بفحص `abs(dfx) < 1e-15` ورفع استثناء `ValueError` عربي صريح ينبه المستخدم دون انهيار التطبيق.

شروط التقارب **س6: ما هو شرط تقارب طريقة النقطة الثابتة $x = g(x)$؟**

الإجابة: شرط نظرية النقطة الثابتة هو أن تكون المشتقة الأولى لدالة التكرار في جوار الجذر بالقيمة المطلقة أقل من الواحد تماماً: $|g'(x)| < 1$. إذا كان $|g'(x)| > 1$ فإن القيم تتباعد (Diverge) نحو اللانهاية، وقد وضعنا فحصاً يحمي من تباعد القيم عند تجاوز 10^{12}.

الأمان البرمجي **س7: كيف قمنا بقراءة وتحليل صيغة الدالة التي يكتبها المستخدم بأمان؟**

الإجابة: تجنبنا تماماً استخدام دالة `eval()` المضمنة لما فيها من ثغرات أمنية خطيرة، واستخدمنا مكتبة `sympy.parse_expr` مع `standard_transformations` و `convert_xor` و `implicit_multiplication_application`، مما أتاح دعم كتابة الأس بـ `^` والضرب الضمني (مثل `3x`) بأمان وتحويلها إلى دالة عددية سريعة بـ `lambdify` واشتقاقها آلياً.